

Movie Review Sentiment Analysis

Project Documentation and Report

Ayush Chaudhary

November 26, 2025

Abstract

This report provides a detailed technical analysis of the Movie Review Sentiment Analysis project. The system leverages Natural Language Processing (NLP) techniques to classify IMDB movie reviews as either positive or negative. Built with Python, Scikit-learn, and Flask, the application demonstrates a complete machine learning pipeline from data preprocessing to web deployment.

Contents

1	Introduction	2
2	System Architecture	2
3	Data Analysis	2
3.1	Dataset Overview	2
3.2	Sentiment Distribution	2
4	Methodology	3
4.1	Text Preprocessing	3
4.2	Feature Engineering	3
4.3	Model Selection	3
5	Application Implementation	3
5.1	Web Interface	3
5.2	File Structure Highlights	4
6	Conclusion	4

1 Introduction

The Movie Review Sentiment Analysis project is an end-to-end NLP application designed to determine the emotional tone behind a series of words, specifically movie reviews. Understanding user sentiment is crucial for businesses and creators to gauge public opinion.

This project utilizes the IMDB dataset containing 50,000 reviews. It implements a Logistic Regression model trained on TF-IDF vectors to predict sentiment with high accuracy. The solution is deployed via a lightweight Flask web application, offering a user-friendly interface for real-time predictions.

2 System Architecture

The system follows a standard Machine Learning pipeline architecture. The data flows from the raw CSV input through preprocessing, vectorization, and model training, resulting in a serialized model pipeline used by the web application.

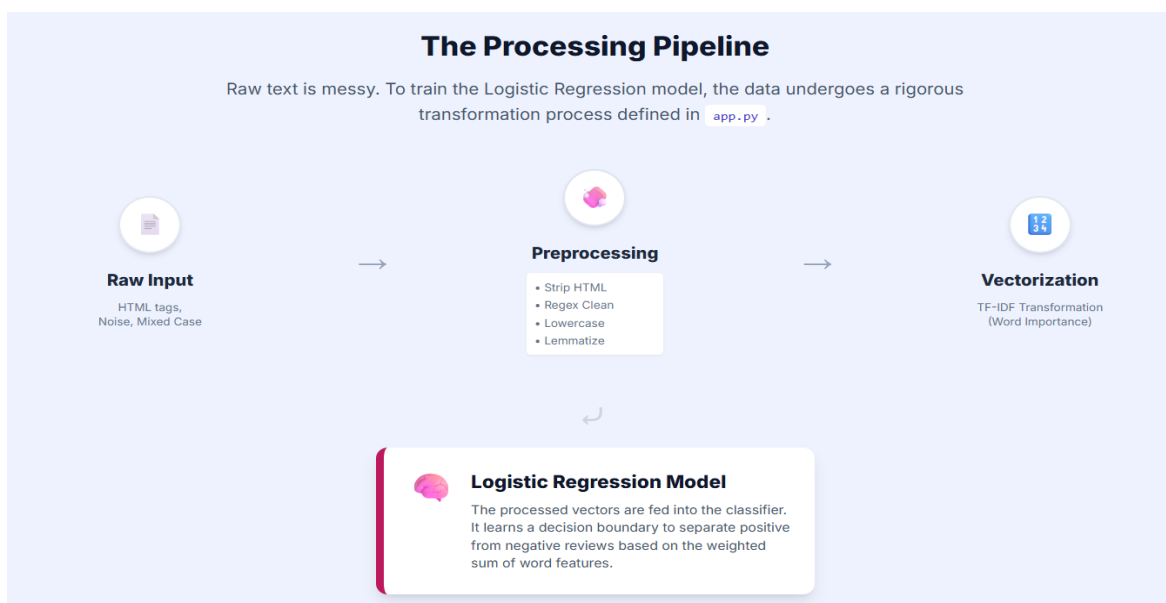


Figure 1: Data Flow and System Architecture Overview

3 Data Analysis

3.1 Dataset Overview

The project uses the IMDB Dataset.csv, a widely used benchmark dataset for binary sentiment classification.

Table 1: Dataset Statistics	
Metric	Value
Total Reviews	50,000
Features	2 (review, sentiment)
Classes	Positive, Negative
Class Balance	Balanced (25k / 25k)

3.2 Sentiment Distribution

The dataset is perfectly balanced, which is ideal for training supervised learning models as it prevents bias toward a majority class.



Figure 2: Distribution of Sentiment Labels in the IMDB Dataset

4 Methodology

4.1 Text Preprocessing

Raw text data is noisy and requires cleaning before analysis. The `preprocess_text` function in the codebase implements the following steps:

- HTML Tag Removal: Uses BeautifulSoup to strip HTML tags (e.g., `<br />`).
- Special Character Removal: Uses Regular Expressions (re) to keep only alphabetic characters.
- Normalization: Converts all text to lowercase to ensure consistency.
- Tokenization & Lemmatization: Splits text into words and reduces them to their base form (lemma) using the WordNetLemmatizer, while removing common English stop words.

4.2 Feature Engineering

The project uses TF-IDF (Term Frequency-Inverse Document Frequency) vectorization. This technique converts text into numerical vectors, highlighting words that are important to a specific document but rare across the corpus, effectively filtering out common but uninformative words.

4.3 Model Selection

A Logistic Regression classifier is employed. Logistic Regression is well-suited for binary classification tasks in high-dimensional sparse spaces (like text data) due to its efficiency and interpretability. The model and vectorizer are bundled into a Pipeline and serialized using joblib.

5 Application Implementation

5.1 Web Interface

The application is exposed via a Flask web server (`app.py`).

- Backend: Handles the /predict POST request, loads the trained .pkl pipeline, and returns JSON responses.
- Frontend: An HTML page styled with Tailwind CSS. It includes JavaScript to handle form submission asynchronously (AJAX) and update the UI with the prediction (Positive/Negative) without reloading the page.

5.2 File Structure Highlights

- train_model.py: Script to train and save the model pipeline.
- analyze.py: Script for Exploratory Data Analysis (EDA).
- requirements.txt: Lists dependencies like flask, nltk, pandas, and scikit-learn.



Figure 3: Expected Model Evaluation Metrics

6 Conclusion

This Movie Sentiment Analysis project demonstrates a robust implementation of a standard NLP workflow. By combining effective preprocessing, TF-IDF vectorization, and a Logistic Regression classifier wrapped in a modern web interface, it provides a practical tool for analyzing textual sentiment.